

POPPER: A Dataflow System for In-Flight Error Handling in Machine Learning Workflows

Adnan Shakeel Ahmed

Abhilash Jindal

Kaustubh Beedkar

Indian Institute of Technology Delhi, India
popper@cse.iitd.ac.in

Abstract—We present POPPER, a dataflow system for building Machine Learning (ML) workflows. A novel aspect of POPPER is its built-in support for in-flight error handling, which is crucial in developing effective ML workflows. POPPER provides a convenient API that allows users to create and execute complex workflows comprising traditional data processing operations (such as map, filter, and join) and user-defined error handlers. The latter enables inflight detection and correction of errors introduced by ML models in the workflows. Inside POPPER, we model the workflow as a reactive dataflow, a directed cyclic graph, to achieve efficient execution through pipeline parallelization. We demonstrate the in-flight error-handling capabilities of POPPER, for which we have built a graphical interface, allowing users to specify workflows, visualize and interact with its reactive dataflow, and delve into the internals of POPPER.

I. INTRODUCTION

Integrating machine learning (ML) algorithms within data analytics has widened the landscape of data-driven applications. At the core of analytics applications lies complex data processing pipelines—often called ML workflows—that interleave ML models with other data transformations as user-defined functions (UDFs) with relational operations such as filter, join, and aggregation.

Example ML Workflow. Figure 1 illustrates a complex ML workflow designed to extract information—such as name, date of birth, and nationality—from scanned images of identity cards. The workflow is represented as a graph, where each node denotes a data transformation, and each edge represents the data flow between nodes. The workflow (for now, ignore the edge labels and the two orange-colored nodes) begins by cleaning the data through image cropping and color channel selection (nodes 2 and 3). Next, it recognizes text in the image using an OCR engine (node 4), classifies each card by type (e.g., Albania ID, Greece ID, etc.; node 5), and applies a type-specific extractor (nodes 6 and 7) to obtain the required information from the OCR output. This extracted information is then combined and written to a CSV file (nodes 10 and 11). Similar workflows involving complex data transformations are being developed across various domains, such as traffic management, debugging autonomous vehicles, and automating tedious manual tasks.

Dataflow Systems and ML Workflow Challenges. Several data processing systems, such as Spark and Flink, have expanded their scope from handling traditional relational workloads to more complex workloads involving UDFs. Although these have allowed for streamlined development and execution

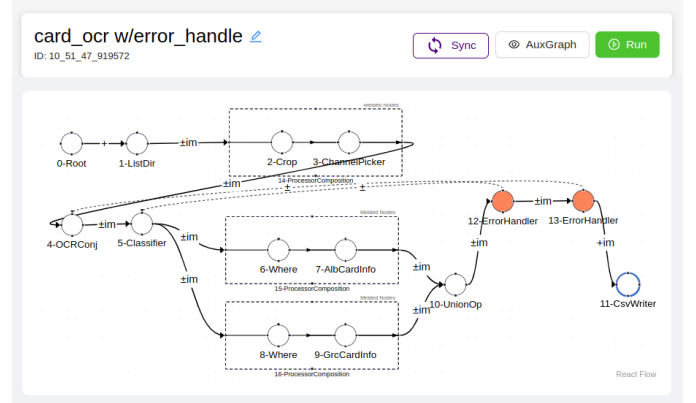


Fig. 1: An example ML workflow developed with POPPER that extracts information from identity card images.

of ML workflows, their effectiveness remains a challenge. This is because the building blocks of ML workflows, which are the ML models, can silently introduce errors. For example, the OCR engine or the classifier in the above example (Fig. 1) may introduce errors due to diverse reasons, including changes in data distribution, incomplete training data, or noisy inputs. ML model errors can, therefore, impact the end-to-end accuracy of ML workflows [2], [3].

POPPER—A Dataflow System for ML Workflows. We have developed a dataflow system called POPPER [1], enabling the specification and efficient execution of ML workflows. The novel feature of POPPER is its built-in support for user-defined error handlers. POPPER provides a Python-based API, allowing domain experts to define complex workflows. Internally, we model the workflow as a *reactive dataflow*, a novel cyclic execution model that enables efficient pipeline-parallel execution of operators. POPPER supports *inflight error handling*—detecting and correcting errors introduced by ML models, which may appear downstream but require upstream correction. For example, in Figure 1, an error (e.g., extracted date of issue being earlier than date of birth) caused by an OCR or classifier may only be evident later. POPPER allows specifying error handlers (nodes 12 and 13) to detect and correct such errors in upstream operators, like using an alternative OCR for erroneous tuples. POPPER then automatically propagates updates downstream, enhancing ML workflow accuracy.

Contributions. In this paper, we demonstrate POPPER using an interactive graphical interface for specifying and executing complex workflows. The purpose of this demonstration is

to a) showcase the effectiveness and efficiency of POPPER by letting the audience specify workflows and user-defined error handlers; b) delve into POPPER internals by letting the audience interact with POPPER and visualize reactive dataflow programs; and c) compare output of workflows in terms of run-time and accuracy with and without error-handlers. The demo interface is available at <https://www.cse.iitd.ernet.in/popper>.

II. SYSTEM

POPPER’s software stack consists of three layers: PYSCOPE, CORE, and EXECUTOR as shown in Figure 2(a). In this section, we outline the functionality and purpose of each layer.

A. PyScope

The top-most layer, PYSCOPE, allows specification of dataflow programs comprising traditional data processing operators, such as `join`, `map`, `filter`, along with two error-handling operators, a `rowErrorHandler` and a `rowSetErrorHandler`. It’s the latter that makes inflight error handling a first-class citizen.

Workflow Specification. PYSCOPE offers a Python-based implementation of the SCOPE language [4] that uses the `rowset` data model and supports user-defined data transformations and error handling. PYSCOPE draws its extensibility from SCOPE’s primitives including `extractor` (for constructing rows from sources like files), `processor` (for row-wise processing), `reducer` (for group-wise processing), `combiner` (for combining rows from two rowsets), and `outputter` (for writing rows to a data sink). Domain experts can easily specify ML workflows by integrating ML model inference in PYSCOPE’s operators. For example, the OCR step in the card workflow shown in Figure 1 (node 4) runs the Tesseract OCR engine on the input image. It is written in PYSCOPE using POPPER’s interface shown in Figure 2(c).

Error Handling. User-defined `RowErrorHandler` allows users to specify three functions: (1) `detect`, which receives a row and outputs true or false depending on whether the error handler found an error; (2) `correct` that receives a row with an error and corrects it by editing the row or any of its ancestor rows, i.e., output rows of upstream operators from which the current row is derived; and (3) `eventually` where the user specifies the behavior of the error handler upon receiving the row again. The user can choose to either `PASS` the row forward or `DROP` the row. For example, the error handler defined in Figure 2(c) checks the consistency of dates extracted from the identity cards. It flags an error if the date of birth is later than the date of issue or if the date of issue is later than the date of expiry (lines 2-4). If it finds an error, it fetches the `img_path` from the upstream operator and then attempts to correct the `ocr` output of the upstream OCR processor by applying a different OCR engine (lines 6-8). Our API includes convenient methods to track the lineage to fetch and correct outputs of upstream operators. We defer discussion on lineage tracking and applying edits to upstream operators to Section II-C. Note that the error handler’s eventual

TABLE I: Edge labels and their semantics.

Property	$R_i + \Delta^+ R_i \rightsquigarrow$	$R_i + \Delta^- R_i \rightsquigarrow$	Label
Non-incremental	$o_j(R_i + \Delta^+ R_i)$	$o_j(R_i + \Delta^- R_i)$	$\bar{i}$
Incremental monotonic			
Only appends	$R_j + \Delta^+ R_j$	$o_j(R_i + \Delta^- R_i)$	im_+
Only deletes	$o_j(R_i + \Delta^+ R_i)$	$R_j + \Delta^- R_j$	im_-
Both	$R_j + \Delta^+ R_j$	$R_j + \Delta^- R_j$	$im_{\pm}$
Incremental non-monotonic			
Only appends	$R_j + \Delta^+ R_j + \Delta^- R_j$	$o_j(R_i + \Delta^- R_i)$	$i\bar{m}_+$
Only deletes	$o_j(R_i + \Delta^+ R_i)$	$R_j + \Delta^+ R_j + \Delta^- R_j$	$i\bar{m}_-$
Both	$R_j + \Delta^+ R_j + \Delta^- R_j$	$R_j + \Delta^+ R_j + \Delta^- R_j$	$i\bar{m}_{\pm}$

behavior is set to `PASS` (line 10), meaning it will forward the erroneous tuple the next time it receives it.

B. Core of POPPER

After a PYSCOPE program has been submitted to POPPER, it is translated into a *reactive dataflow program*, which is a directed cyclic graph of operators. Furthermore, PYSCOPE’s compiler derives edge properties (discussed below) that are later exploited for plan optimization and execution. The CORE takes as input a reactive dataflow, a directed cyclic graph of operators and dataflow edges and produces an execution plan amenable for pipeline parallel execution of plan operators.

Reactive Dataflow. We model an ML workflow with error handlers as a directed cyclic graph, referred to as *reactive dataflow*. Our programming model, thus, allows data to “flow” back to an upstream operator. This is crucial for inflight error handling, where an error in the dataflow might only be evident in some downstream operator’s output (as in the example in Figure 1). More formally, we denote a reactive dataflow as a graph $G = (\mathcal{O}, \mathcal{E}_f \cup \mathcal{E}_b)$, where $\mathcal{O}$ is the set of plan operators (vertices) and $\mathcal{E}_f$ and $\mathcal{E}_b$ are sets for forward and backward dataflow (edges). While a forward edge connects an upstream operator to a downstream operator (as in regular directed acyclic dataflow programs), a backward edge connects a downstream (error handler) operator to an upstream operator. For example, Figure 1 shows a reactive dataflow, where `DateErrorHandler` operators are error handlers.

Edge Labels. Each backward edge in the reactive dataflow carries a label, which captures the semantics of how the error handler modifies the output of the upstream operator. An error handler may append (denoted by $+$) and/or delete (denoted by $-$) the output (rows) of an upstream operator. In the reactive dataflow in Figure 1 for example, `13-DateErrorHandler` edits output rows ($\pm$) of operator `5-Classifer`. Conversely, each forward edge carries a label, which captures the transformation properties of the operators concerning handling changes in their inputs.

Edge Properties. We categorize forward edges’ labels as either non-incremental (denoted by $\bar{i}$) or incremental (denoted by i), and further categorize incremental edges as either incremental and monotonic (denoted by im) or incremental and non-monotonic (denoted by $i\bar{m}$). In more detail, consider a forward edge $o_i \rightarrow o_j$. We say that a forward edge is *non-incremental* if a change $\Delta^{op} R_i$, $op \in \{+, -\}$, in input of

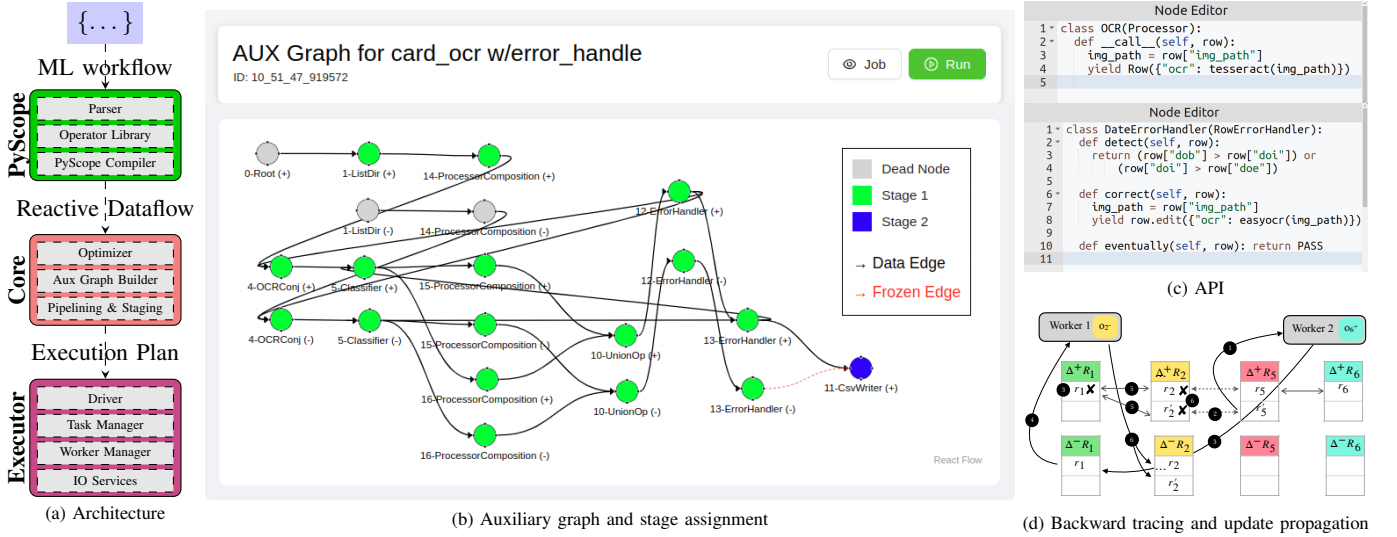


Fig. 2: Overview of POPPER.

o_j (output of o_i) leads recomputing o_j . For example, if o_j is a stateless min operator. Additionally, we say that a forward edge is *incremental and monotonic* if an append (delete) $\Delta^+ R_i$ ($\Delta^- R_i$) in the input of o_j leads to “incremental” appends (deletes) $\Delta^+ R_j$ ($\Delta^- R_j$) in the output R_j of o_j . For example, if o_j is a map operator. Likewise, we say that a forward edge is *incremental and non-monotonic* if an append ($\Delta^+ R_i$) or delete ($\Delta^- R_i$) in the input of o_j leads to both “incremental” appends and deletes ($\Delta^+ R_j + \Delta^- R_j$) in the output R_j of o_j . For example, when deleting a row in the right input of a left-join. It is worth noting that incremental edges’ monotonic and non-monotonic behavior also depends on the type (+, -) of update on the input. Table I summarizes the forward edge labels and their semantics. Our above categorization enables us to efficiently and effectively propagate inflight updates downstream: we defer to the discussion below on how we exploit forward edge labels to determine an efficient execution plan. Moreover, it offers a principled approach to formalize notions of incremental computations.

Auxiliary Graph. To enable pipeline parallel execution, CORE organizes dataflow into stages, where each stage represents a set of operators that can be pipelined, i.e., can be executed in parallel. To do so, it first creates an *auxiliary graph* of the reactive dataflow and identifies the blocking and non-blocking behavior of operators with respect to handling changes in their inputs. More specifically, the auxiliary graph follows from the reactive dataflow by applying transformation rules, which for each edge $o_i \xrightarrow{\ell} o_j \in \mathcal{E}_f \cup \mathcal{E}_b$, depending on its label ℓ , explicitly creates append ($\Delta^+ R_i$ and $\Delta^+ R_j$) and delete ($\Delta^- R_j$, $\Delta^- R_j$) streams consumed by operator instances o_j^+ (that appends) and o_j^- (that deletes) rows in its output R_j . Further, based on the edge properties described in the table above, it annotates edges in the auxiliary graph as either blocking or non-blocking. Figure 2(b) shows the auxiliary graph obtained for the reactive dataflow in Figure 1, where gray nodes are marked as dead nodes, which do not consume

any dataflow, and solid black and dashed red edges denote non-blocking and blocking behavior of operators w.r.t. to changes in their input.

Operator Pipelining & Stages. After constructing the auxiliary graph, POPPER decomposes the dataflow into stages, enabling operators within each stage to pipeline data. Operators in a stage may contain cycles due to backward edges from error handlers. For instance, Figure 2(b) shows two stages, with nodes in the same stage sharing identical colors. Stages execute sequentially based on dependencies—for example, stages 1 and 2 are executed first, followed by stage 3, and finally stage 4. For more details on POPPER’s internals and a formal treatment of this approach, see [1].

C. Executor

We now briefly discuss POPPER internals concerning cyclic dataflow execution, which involves inflight updates to upstream operators’ output and propagating it downstream.

Dataflow Execution. POPPER’s executor manages dataflow execution by having its worker manager spawn a stateless driver and worker processes for computation. It coordinates processes through shared states in an in-memory data store. For each operator instance o_i^+ and o_i^- , POPPER keeps an output stream $\Delta^+ R_i$ and a deletion stream $\Delta^- R_i$ in the data store. When executing an operator, workers read from one or more streams of parent operators as defined by the auxiliary graph, and write to the operator’s output and deletion streams.

Backward Tracing and Update Propagation. To efficiently correct in-flight errors detected by a downstream error handler in an upstream operator, POPPER keeps instance-level lineage pointers in its in-memory data store. Each row has two pointer sets: backward pointers for identifying immediate parents (used in backward tracing) and forward pointers for immediate children (used to incrementally propagate deletions, as shown in Fig. 2(d)). For further details on POPPER’s execution, backward tracing, and update propagation, see [1].

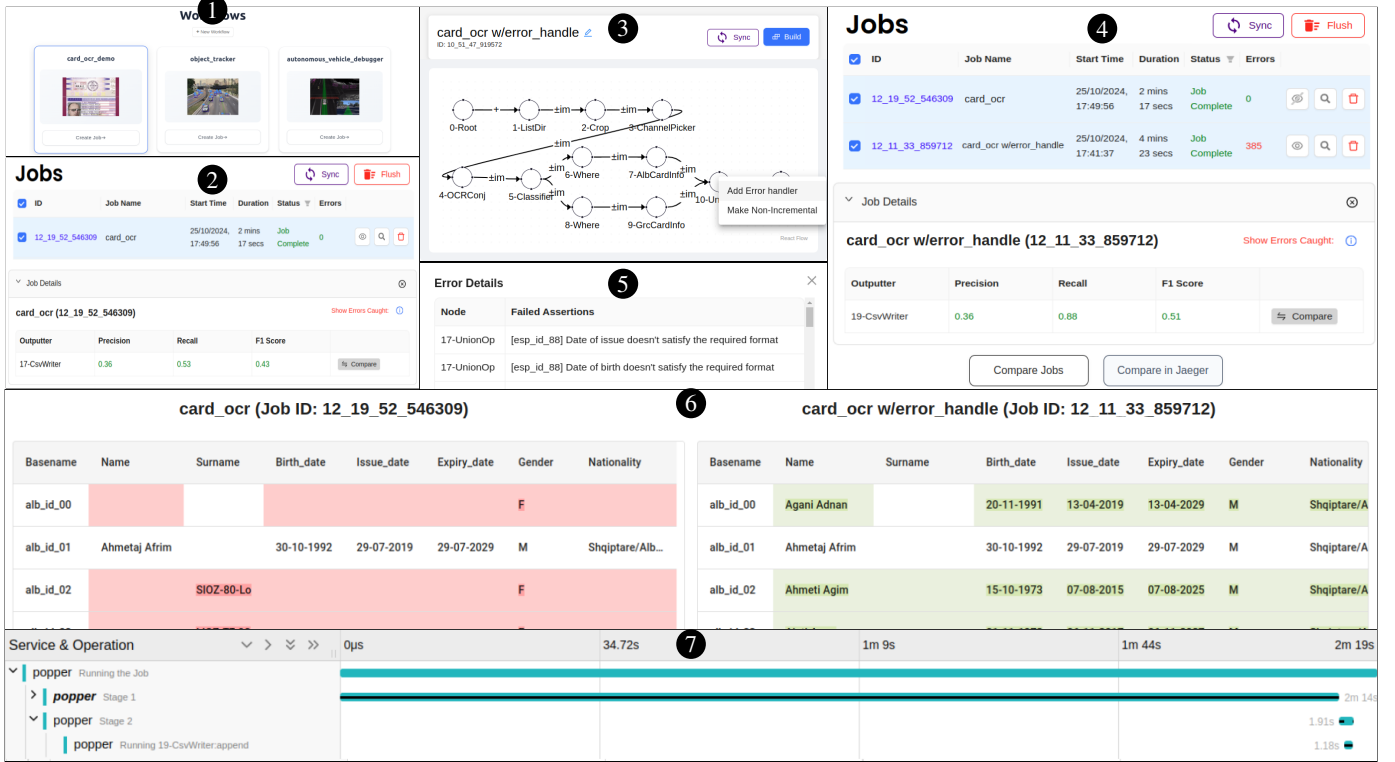


Fig. 3: POPPER's graphical interface.

III. DEMONSTRATION SCENARIOS

We now present the demonstration scenarios (Figure 3) available to the attendees. The demo interface is available at <https://www.cse.iitd.ernet.in/popper>. Our demonstration focuses on (1) showcasing the ease of writing ML workflows and error handlers, (2) demonstrating the effectiveness of error handlers, and (3) exploring POPPER's internals.

End-to-End Workflow Execution. Attendees will have access to three predefined ML workflows: one based on Figure 1, along with traffic analysis and autonomous vehicle debugging ①. They can also create a new workflow by selecting the new workflow button. Through the graphical interface, attendees can build and run workflows. The Job page displays all running and completed workflows ②, with completed workflows showing key statistics like runtime, precision, recall, and the number of errors corrected by each error handler.

Interactive Error Handling. Attendees can interact with the reactive dataflow to add error handlers (by clicking on an edge) or delete existing error handlers by clicking error handlers ③. For instance, after choosing to add an error handler, attendees will experience the error handling and the editing PYSCOPE API provided by POPPER (Figure 2(c)).

Effectiveness of Error Handling. Attendees can compare the final output with the ground truth to design custom error handlers. By running different workflow versions with various error handlers, they can observe the impact on runtime, precision, and recall ④. They can also explore the individual errors detected by each error handler ⑤. Additionally, attendees

can select and compare two workflows ⑥ to assess the fine-grained impact on individual data tuples when enabling or disabling specific error handlers.

Popper Internals. The interface also allows attendees to interact with the reactive dataflow (Figure 1), build the workflow to see its auxiliary graph and the corresponding stage assignment (Figure 2(b)), and finally run the workflow. Further, attendees can interactively change the edge properties and visualize changes in the reactive dataflow ③. We note that attendees can only change incremental edges to non-incremental ones. Finally, attendees can also view a fine-grained trace to see the stage-based pipeline execution of POPPER in action ⑦.

ACKNOWLEDGEMENTS

We thank the anonymous reviewers for their feedback. This research was supported in part by R Systems Center of Excellence (CoE) on Applied AI for Sustainable Systems at IIT Delhi.

REFERENCES

- [1] A. Jindal, K. Beedkar, V. Singh, J. N. Mohammed, T. Singla, A. Gupta, and K. Choudhary, "Reactive dataflow for inflight error handling in ml workflows," in *Proc. of the Eighth Workshop on Data Management for End-to-End Machine Learning, DEEM@SIGMOD*, 2024, p. 51–61.
- [2] D. Kang, D. Raghavan, P. Bailis, and M. Zaharia, "Model assertions for monitoring and improving ml models," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 481–496, 2020.
- [3] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, "Operationalizing machine learning: An interview study," 2022.
- [4] R. Chaiken, B. Jenkins, P.-Å. Larson, B. Ramsey, D. Shakib, S. Weaver, and J. Zhou, "Scope: easy and efficient parallel processing of massive data sets," *Proc. of VLDB endowment*, vol. 1, no. 2, pp. 1265–1276, 2008.